



Apache Kafka — Day 4

Stream Processing, ksqldb, Schema Registry & Confluent Control Center

8 HOURS

INTERMEDIATE → ADVANCED

CONCEPT → ARCHITECTURE → QUERY → SCHEMA → DEMO → HANDS-ON

Day 4: From Event Transport to Event Processing

1

Day 1

Kafka Fundamentals & Architecture

2

Day 2

Producer / Consumer & Topic Management

3

Day 3

Security / Operations / Kafka Connect

4

Day 4

Stream Processing + Schema Management

Today's Learning Objectives

Stream Processing

Concepts & Kafka
Streams architecture

KStream & KTable

Framework selection &
operations

Schema Registry

Avro schemas,
subjects & versions

Compatibility

Schema evolution &
compatibility rules

ksqlDB

Streams, tables, joins, REST API & monitoring

Control Center

Confluent UI-based operations

SECTION 2

What Is Stream Processing?

Traditional applications process data **after** it is stored. Stream processing works with events **as they continuously arrive**.



Fraud Detection

Detect suspicious transactions in real time



Live Sales

Calculate revenue as orders arrive



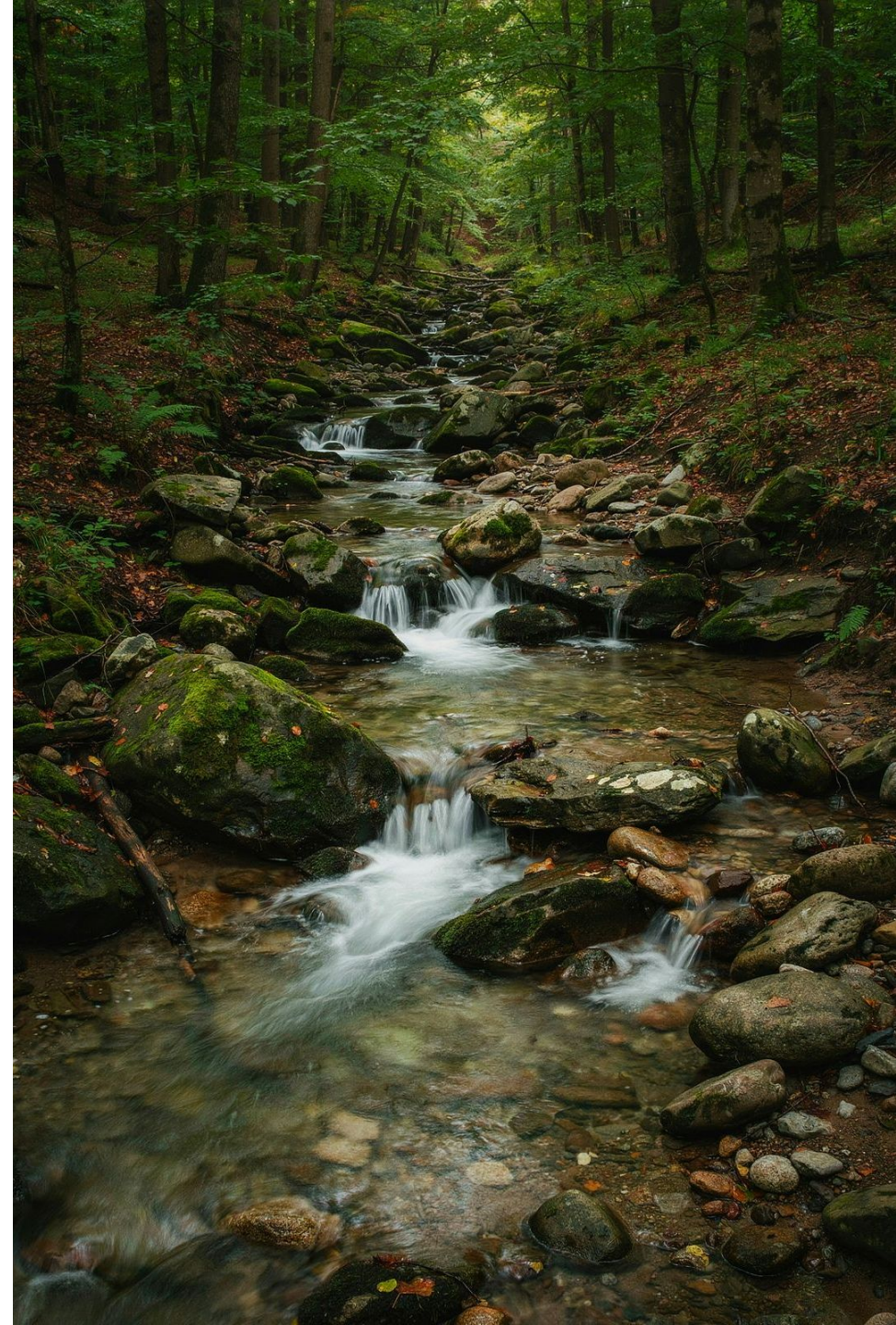
Order Tracking

Track status changes continuously

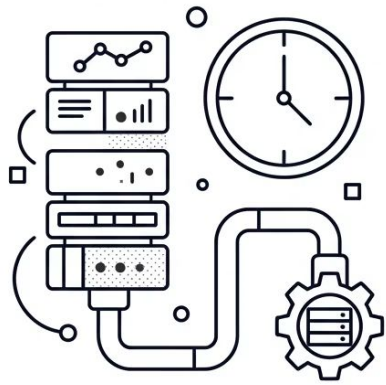


Real-Time Alerts

Generate alerts from IoT & event streams

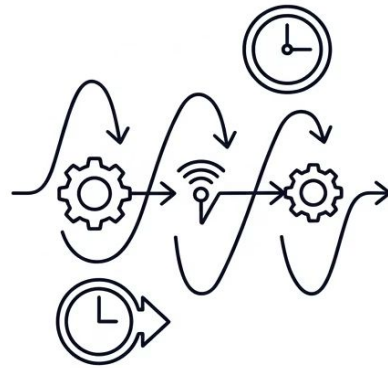


Batch vs. Stream Processing



BATCH

- BOUNDED DATASET
- PERIODIC PROCESSING
- MINUTES/HOURS DELAY
- HISTORICAL ANALYSIS
- SCHEDULED JOBS



STREAM

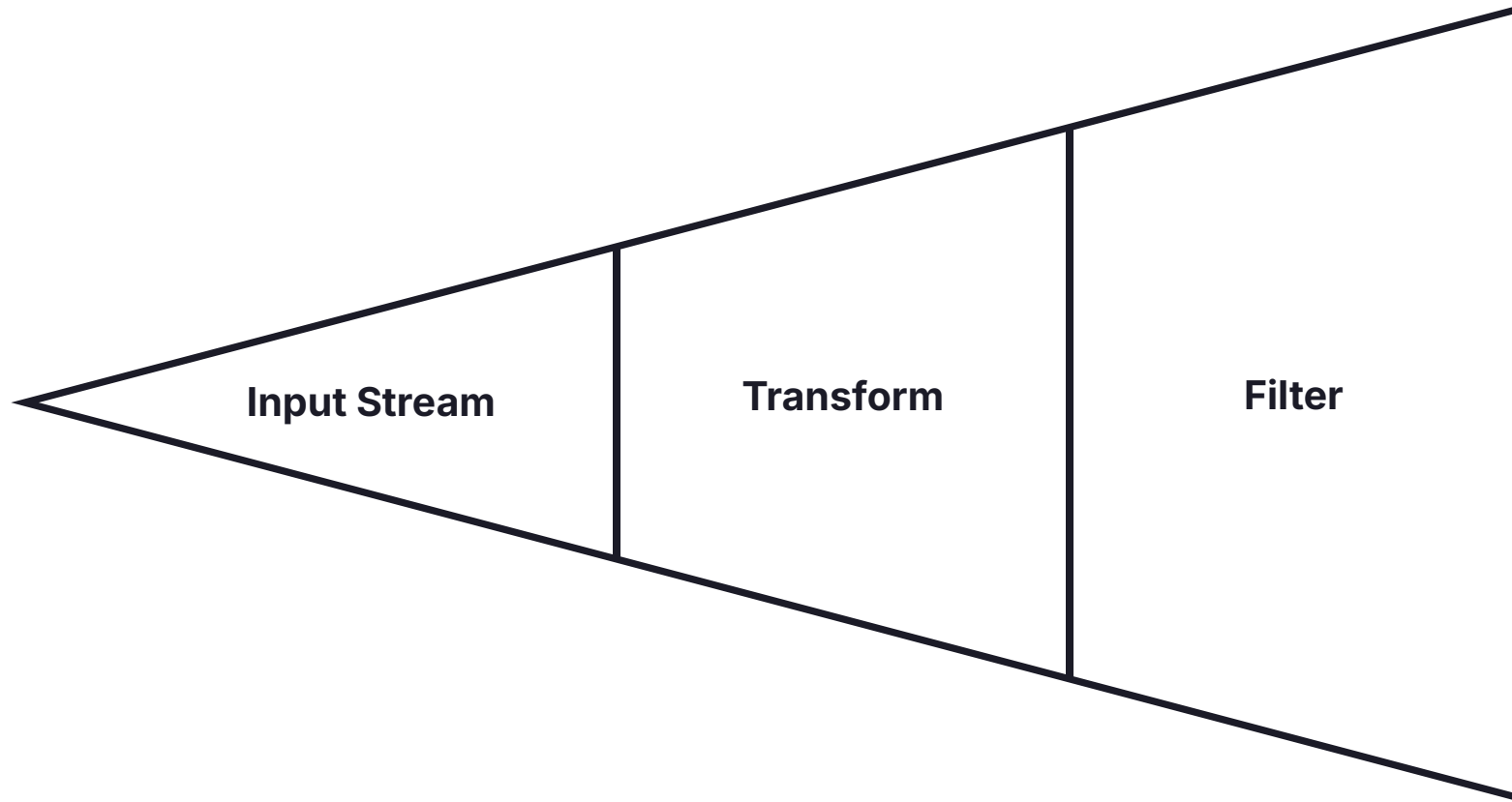
- CONTINUOUS EVENT FLOW
- CONTINUOUS PROCESSING
- LOWER LATENCY
- REAL-TIME ANALYSIS
- CONTINUOUSLY RUNNING JOBS

Banking Example

Batch: Transactions → End-of-Day Job → Fraud Report

Streaming: Transaction → Kafka → Fraud Processing → **Immediate Alert**

Stream Processing Pipeline



Example: payment-events → Filter high-value payments → Enrich with customer info → Calculate totals → high-value-payment-events

SECTION 3

What Is Kafka Streams?

Kafka Streams is an **application-oriented library** for processing records directly from Kafka topics — no separate cluster required.

Input

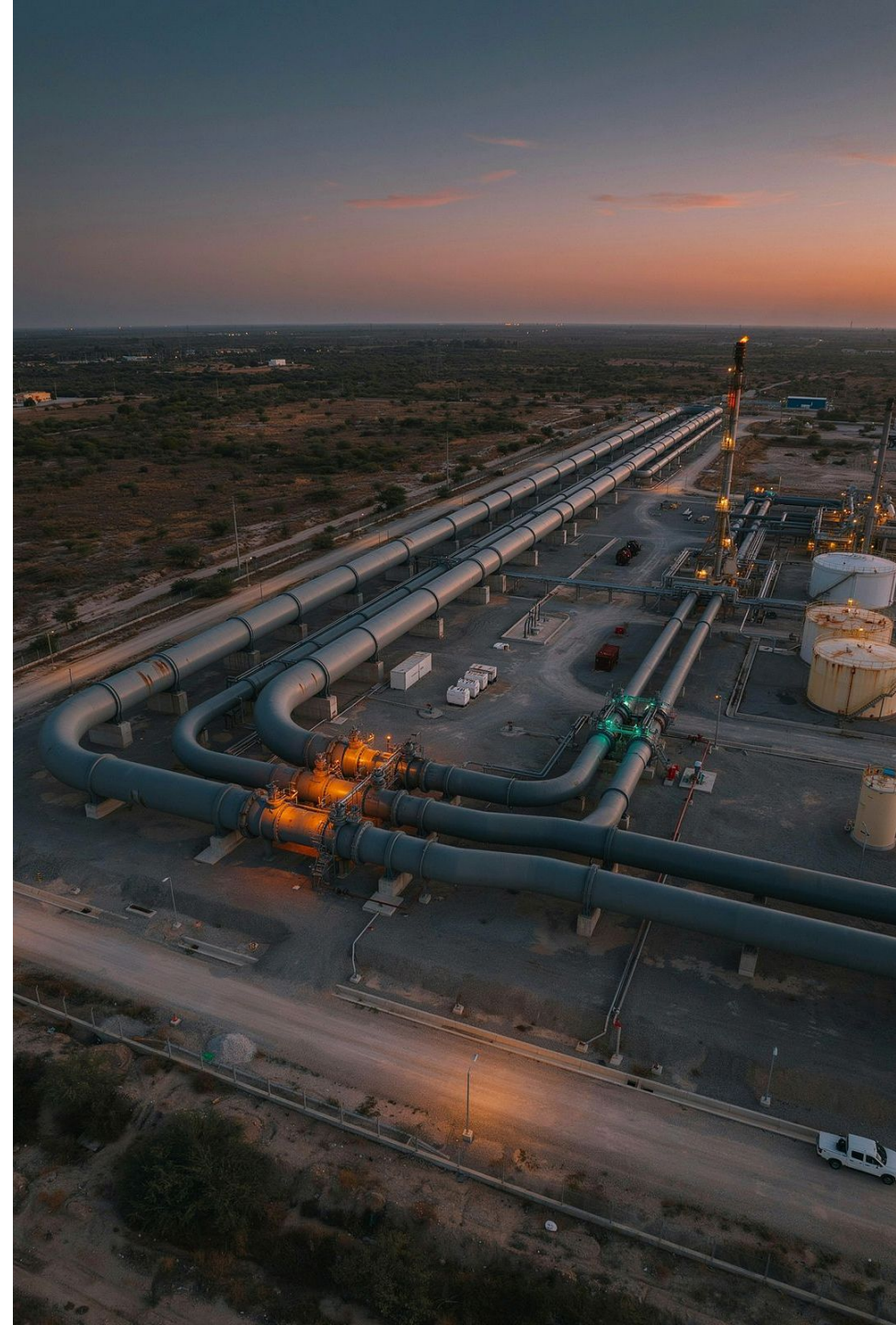
Reads from Kafka Topics

Process

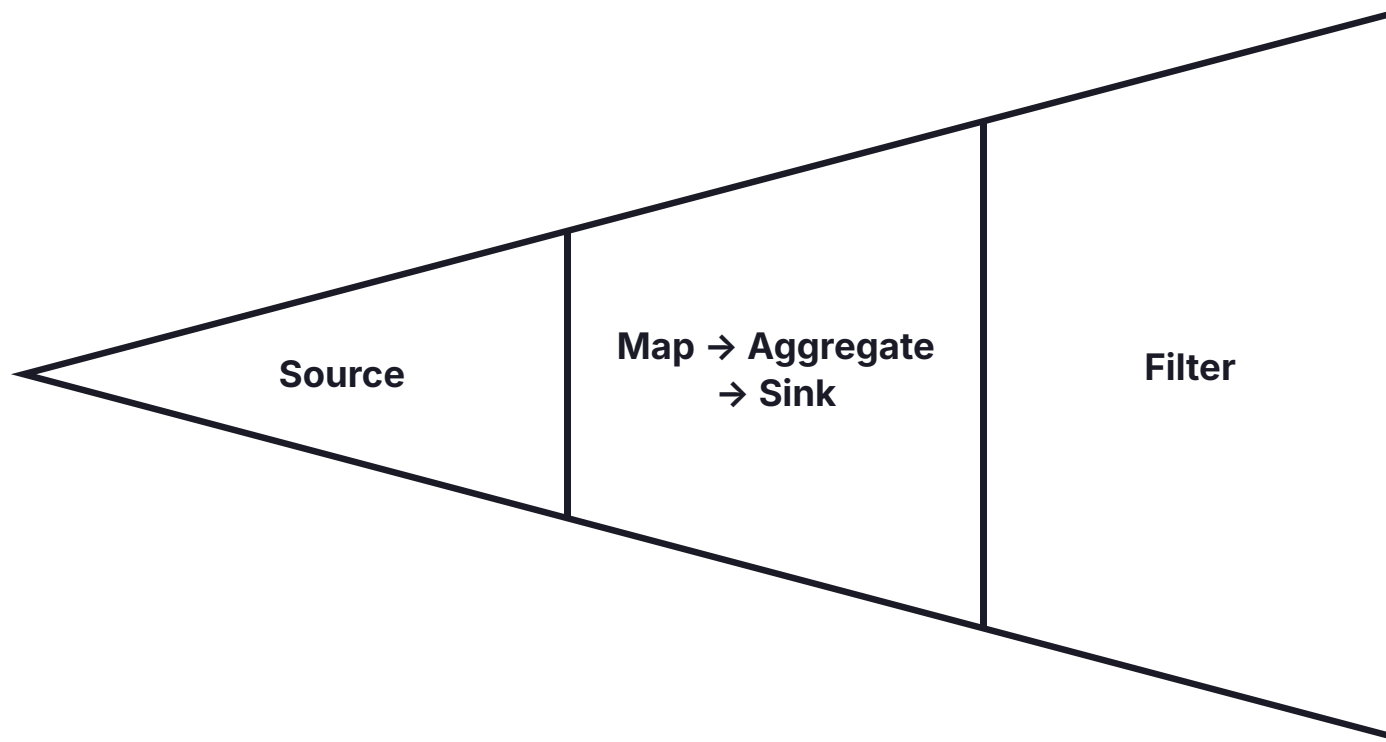
Filter · Transform · Aggregate
· Join

Output

Writes back to Kafka Topics



Kafka Streams Architecture



Processing Topology

Each operation in a Kafka Streams application forms a node in a **processing topology** — a directed graph from source to sink.

- order-events → Filter → Map → Aggregate → processed-orders
- Each step is composable and independently scalable

SECTION 3

Choosing a Stream Processing Framework

Data Source

Is Kafka the primary event backbone?

Processing Complexity

Simple filtering or complex stateful logic?

Deployment Model

Embedded library vs. separate platform?

Development Model

Code-based (Kafka Streams) vs. SQL-style (ksqlDB)?

Understanding KStream & KTable

KStream — Event Stream

Every record is a **new event**. History is preserved.

T1 → Account101 → ₹500	T2 → Account101 → ₹1,000
T3 → Account102 → ₹750	

KTable — Current State

Records represent **updates to state**. Only the latest value matters.

Customer101 → GOLD (updated)	Customer102 → GOLD
------------------------------	--------------------

📌 **Mental Model:** KStream = Events/Changes · KTable = Current State View

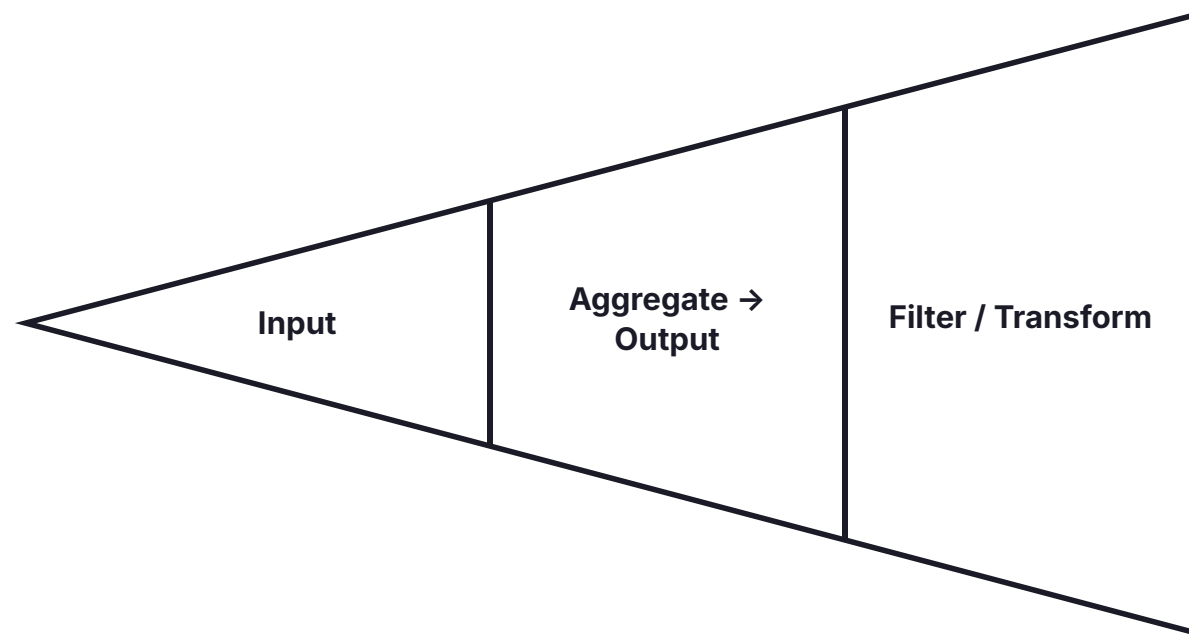
Filtering, Transforming & Aggregating Streams

Filter Example

Transaction stream filtered by **amount** > ₹100,000 passes only ₹150,000 and ₹200,000 through.

Aggregation Example

Group by account key → sum amounts continuously:



Account101

₹500 total

Account102

₹500 total



SECTION 6

KStream-KTable Join

Enrich a **transaction event** with **customer state** by joining a KStream with a KTable.

1

KStream

transaction-events: accountId: 101, amount: ₹150,000

2

KTable

customer-profile: Risk = HIGH, Segment = PREMIUM

3

JOIN

enriched-transactions: T101 · ₹150,000 · HIGH · PREMIUM

Why ksqlDB?

SQL-Style Stream Processing

Analysts and developers can process Kafka streams **without writing a full application** — using familiar SQL syntax instead.

Kafka Streams API	ksqlDB
KStream	STREAM
KTable	TABLE
Code-based	SQL-based



Creating Streams & Queries in ksqlDB

Define a Stream

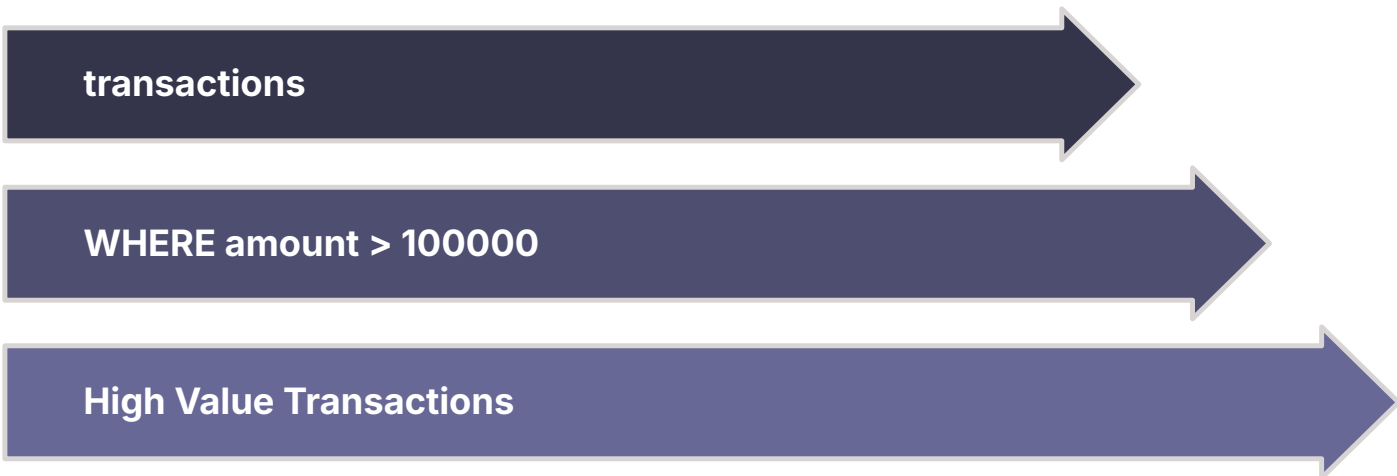
```
CREATE STREAM transactions (  
  transaction_id VARCHAR,  
  account_id   VARCHAR,  
  amount       DOUBLE  
);
```

Query It

```
SELECT * FROM transactions  
WHERE amount > 100000;
```

Live Results

Run a producer while the query executes — participants see **high-value transactions appear continuously** in real time.



The diagram consists of three horizontal arrows pointing to the right, stacked vertically. The top arrow is dark blue and contains the text 'transactions'. The middle arrow is a slightly lighter shade of blue and contains the text 'WHERE amount > 100000'. The bottom arrow is a medium blue and contains the text 'High Value Transactions'.

transactions

WHERE amount > 100000

High Value Transactions

Joining Streams & Tables in ksqlDB

```
SELECT t.transaction_id, t.amount, c.customer_name
FROM transactions t
JOIN customers c
ON t.account_id = c.account_id;
```

1**STREAM**

transactions — continuous events

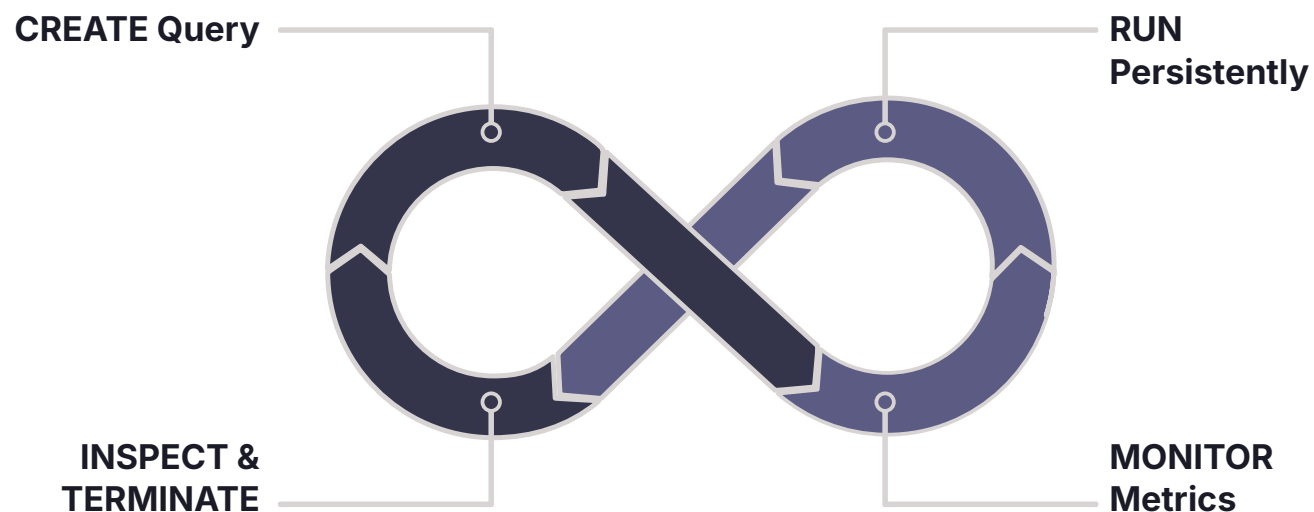
2**TABLE**

customers — current state

3**OUTPUT**

enriched_transactions — joined
result

Persistent Queries & Monitoring

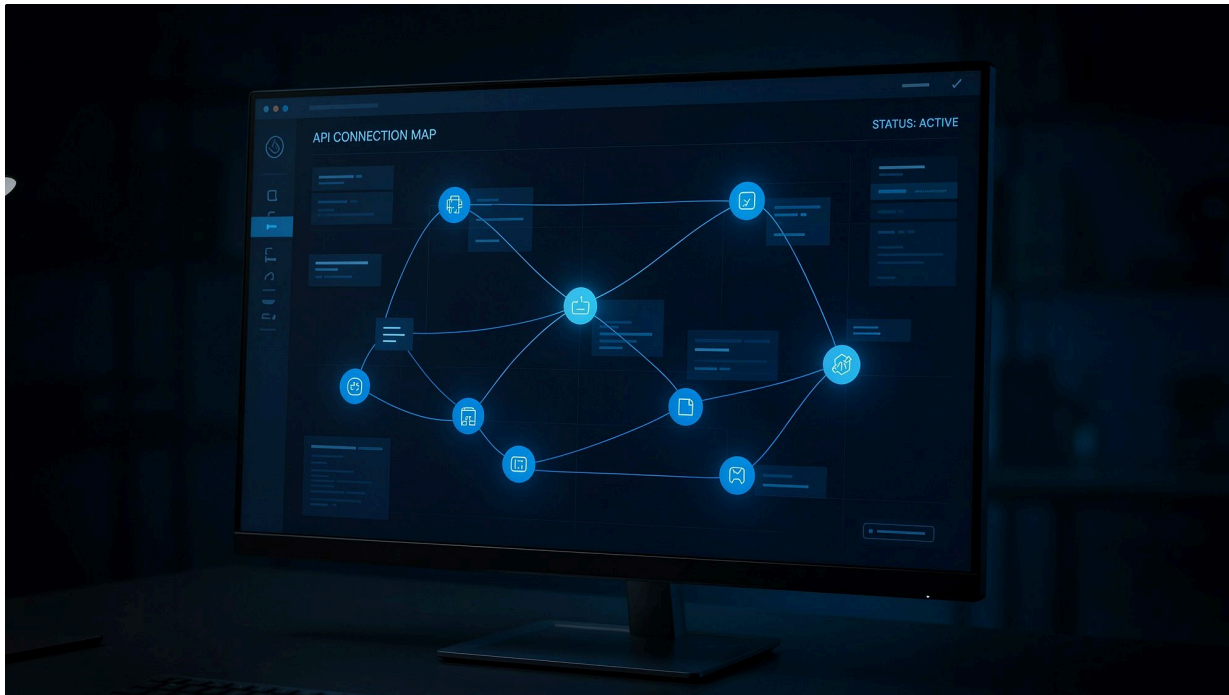


Key Concept

Persistent queries are **continuously executing workloads** — not one-time SQL statements. They run until explicitly terminated.

- Input Topic → Persistent Query → Output Topic
- Monitor throughput, lag, and errors
- Inspect query state at any time

ksqlDB REST API



Programmatic Access

Applications interact with ksqlDB over **HTTP / REST** — no direct Kafka client required.

Submit SQL

Execute stream operations via API

Integrate

Embed streaming into enterprise apps

Manage

Query & control ksqlDB programmatically

Hands-on Lab 1: Stream Processing with ksqlDB

01

Create Stream

Define **transactions** stream in ksqlDB

02

Produce Events

T101→A101→₹5,000 ·
T102→A102→₹150,000 ·
T103→A101→₹250,000

03

Filter & Query

Create and run high-value transaction query

04

Create Table & Join

customer-profile table + Transaction Stream → Enriched Stream

05

Monitor & Terminate

Inspect running query, then terminate cleanly

✅ Outcome: End-to-end streaming transformation — no conventional producer/consumer application required.

The Data Contract Problem

A producer changes its event structure. An older consumer still expects the original field `name` — but now receives `fullName`.

1

Producer Change

Adds `fullName`, `email`; removes `name`

2

Contract Mismatch

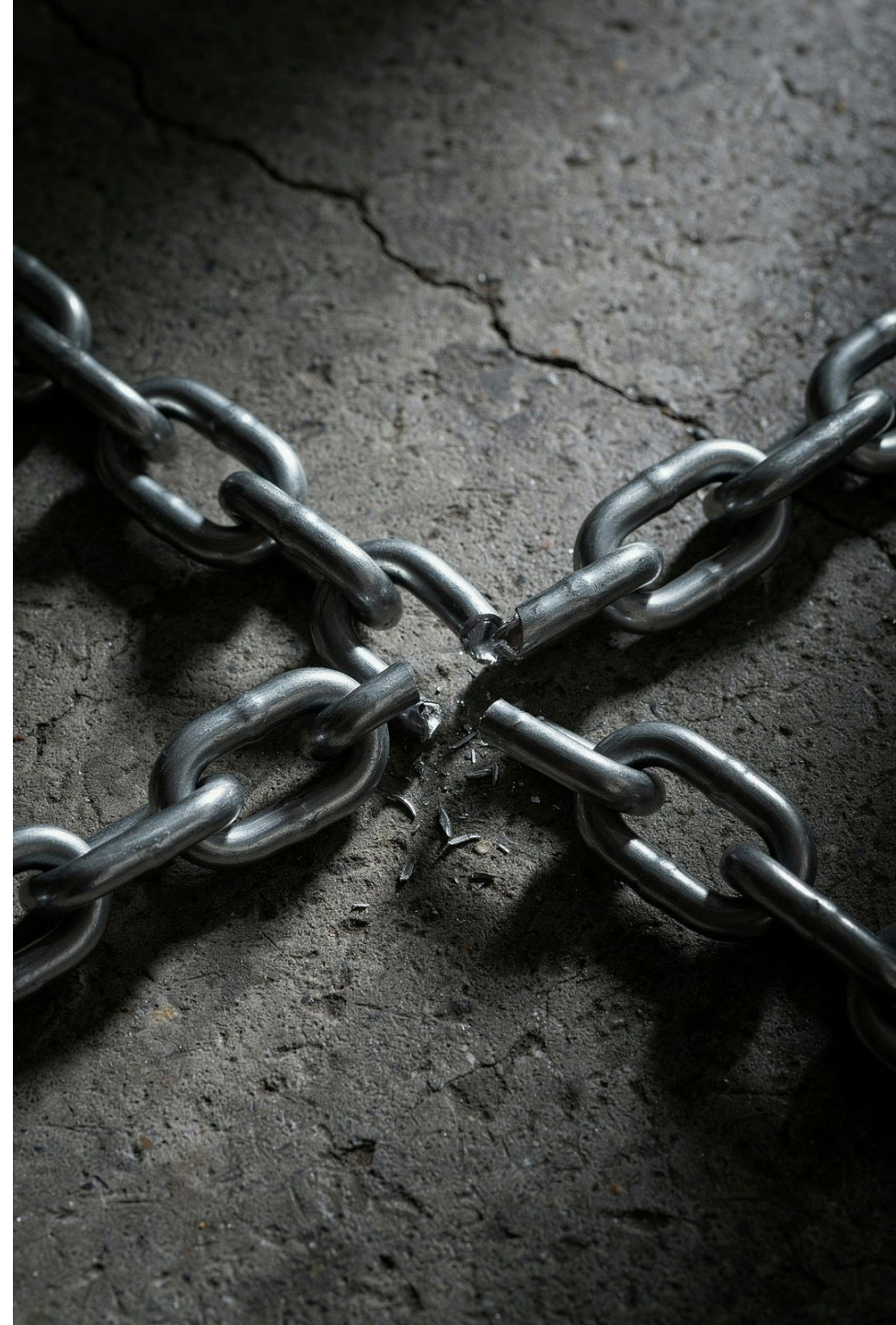
Consumer expects field that no longer exists

3

Consumer Failure

Silent data loss or runtime error

Enterprise question: How do we allow schemas to evolve without unexpectedly breaking consumers?



Schema as a First-Class Citizen

What Is a Schema?

A schema defines the **expected structure** of data — an explicit contract rather than an assumption buried in code.

PaymentEvent Schema

```
transactionId : string
accountId    : string
amount       : double
currency     : string
timestamp    : long
```

Clear Contracts

Producers & consumers agree on structure

Validation

Reject malformed events at the source

Controlled Evolution

Change schemas safely over time

Avro Schema Example

```
{  
  "type": "record",  
  "name": "PaymentEvent",  
  "fields": [  
    {"name": "transactionId",  
      "type": "string"},  
    {"name": "accountId",  
      "type": "string"},  
    {"name": "amount",  
      "type": "double"}  
  ]  
}
```



Why Not Just Use Unmanaged JSON?

Plain JSON is easy to start with — but without a managed contract, critical questions go unanswered:

Type Safety

Is `orderId` a string or number?
Can `amount` be null?

Required Fields

Which fields are mandatory vs. optional?

Evolution

Can fields be renamed? Which version does a consumer expect?

📌 The lesson: enterprise event contracts need **deliberate governance** — not just a convenient format.



SECTION 12

What Is Schema Registry?

A **central service** that stores, versions, and enforces schemas for Kafka data — the safeguard around schema management.



Store Schemas

Central repository for all event schemas



Manage Versions

Track schema evolution over time



Compatibility Checks

Validate changes before deployment



Subjects

Organize schemas by logical grouping

Schema Registry Terminology

Schema

Definition of the data structure

Subject

Logical name under which schemas are registered (e.g. `payment-events-value`)

Version

Evolution of a schema over time — V1, V2, V3...

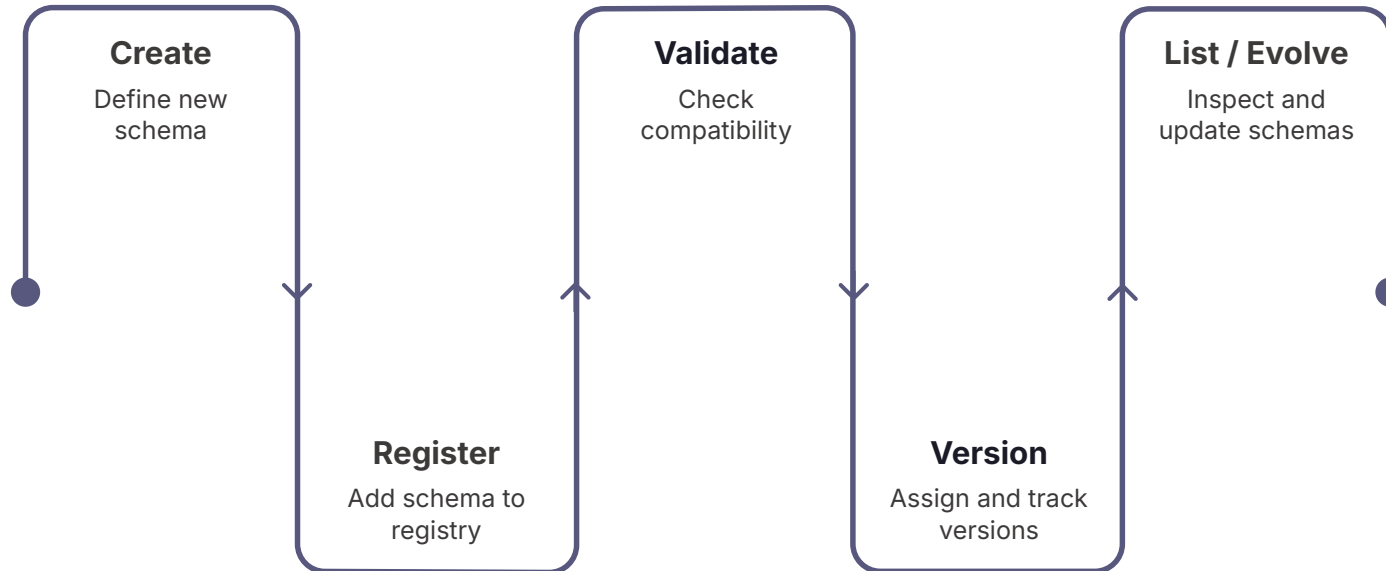
Subject Hierarchy

Subject: `payment-events-value`

- Version 1
- Version 2
- Version 3

This structure provides **history and governance** for every schema change.

Schema Registry Operations



Typical Operational Questions

- Which subjects exist?
- What is the latest schema version?
- Is a proposed change compatible?
- Which schema version is the application using?

Operations include: **Add** · **Delete** · **Validate** · **List Subjects** · **Inspect Versions**

Why Schema Compatibility Matters

Producer and consumer may not upgrade simultaneously. A V2 producer writes data that a V1 consumer must still read.

V1 Schema

transactionId · amount

V2 Schema

transactionId · amount · **currency** (new field)

Can applications using **different schema versions** continue to work together? This is the purpose of **compatibility rules**.

Schema Compatibility Concepts

Backward

New schema readers work with data produced by **previous** schema versions

Forward

Older readers work with data produced by **newer** schema versions

Full

Combines both backward and forward requirements

📌 **Architecture lesson:** Schema evolution should be Designed → Validated → Registered → Deployed — not "change producer and hope consumers work."

End-to-End Schema-Aware Flow



This connects Day 3's Avro introduction with Day 4's schema-management concepts — producers and consumers both coordinate through **Schema Registry**.



SECTION 16

Confluent Control Center

A graphical interface for observing and operating the Confluent Kafka environment.



Topics

Browse, inspect, and manage topics and messages



Consumers

Monitor consumer groups and lag



Streams & Queries

Observe ksqlDB persistent queries



Training objective: students should be comfortable with **both** CLI/API and UI-based operations.

Hands-on Lab 2: Schema Registry & Avro

01

Define Schema

Create PaymentEvent: transactionId · accountId · amount

02

Register & Inspect

View Subject · Version · Schema in Registry

03

Avro Producer

Serialize objects → Kafka via Avro

04

Avro Consumer

Deserialize Kafka messages → objects

05

Schema Evolution

Introduce a controlled change and validate compatibility

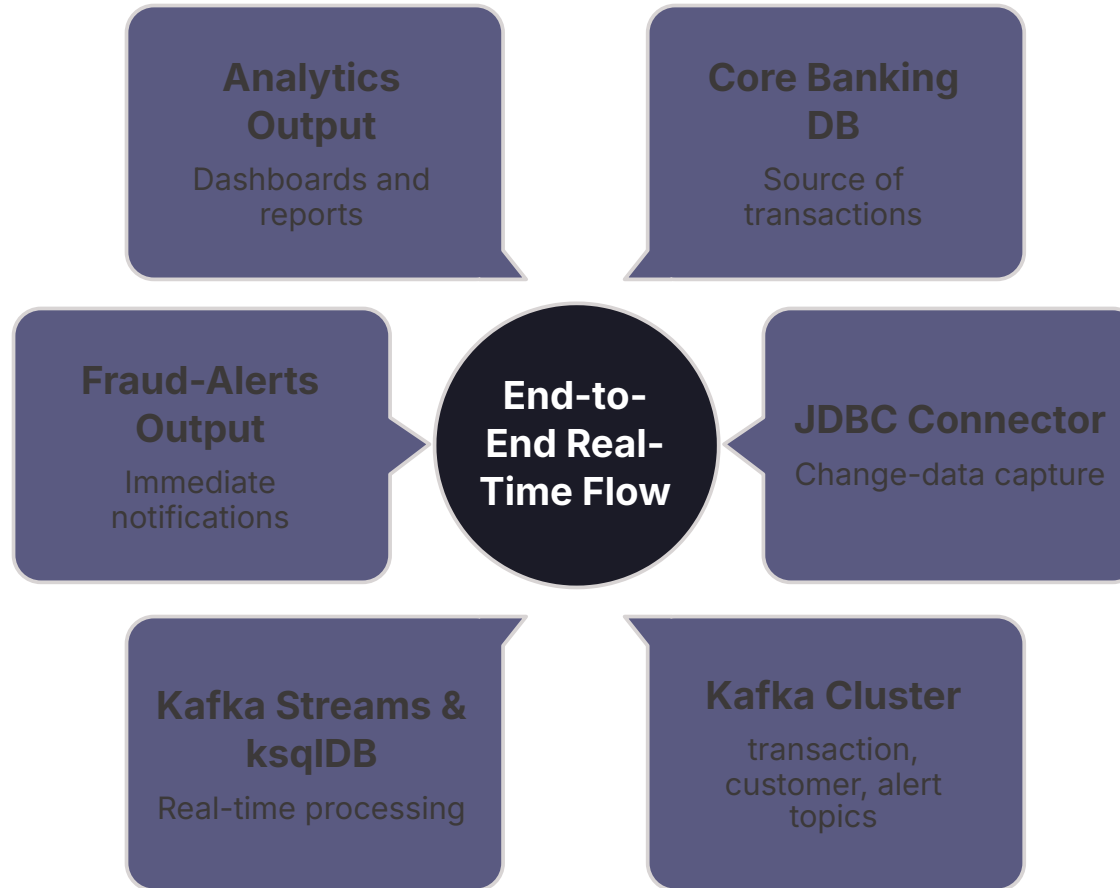
06

UI Exploration

Explore Schema Registry & Control Center operations

End-to-End Real-Time Banking Platform

Bringing all four days together into a single enterprise architecture.



Full Platform: All Layers

Ingestion

JDBC Connector pulls from Core Banking DB into Kafka

Processing

Kafka Streams → Fraud Processing · ksqlDB → High-Value Transactions & Analytics

Schema

Schema Registry enforces Avro event contracts across all topics

Security

Authorization · ACL · SSL across the entire cluster

Day 4 — Key Takeaways

1 Stream Processing

Process events continuously as they arrive — not after storage

3 `ksqlDB`

SQL-style stream processing with persistent queries and REST API

2 KStream vs. KTable

Events vs. current state — join them to enrich data in motion

4 Schema Registry

Govern Avro schemas with subjects, versions, and compatibility rules

What's Next?

Practice

Complete both hands-on labs end-to-end

Explore

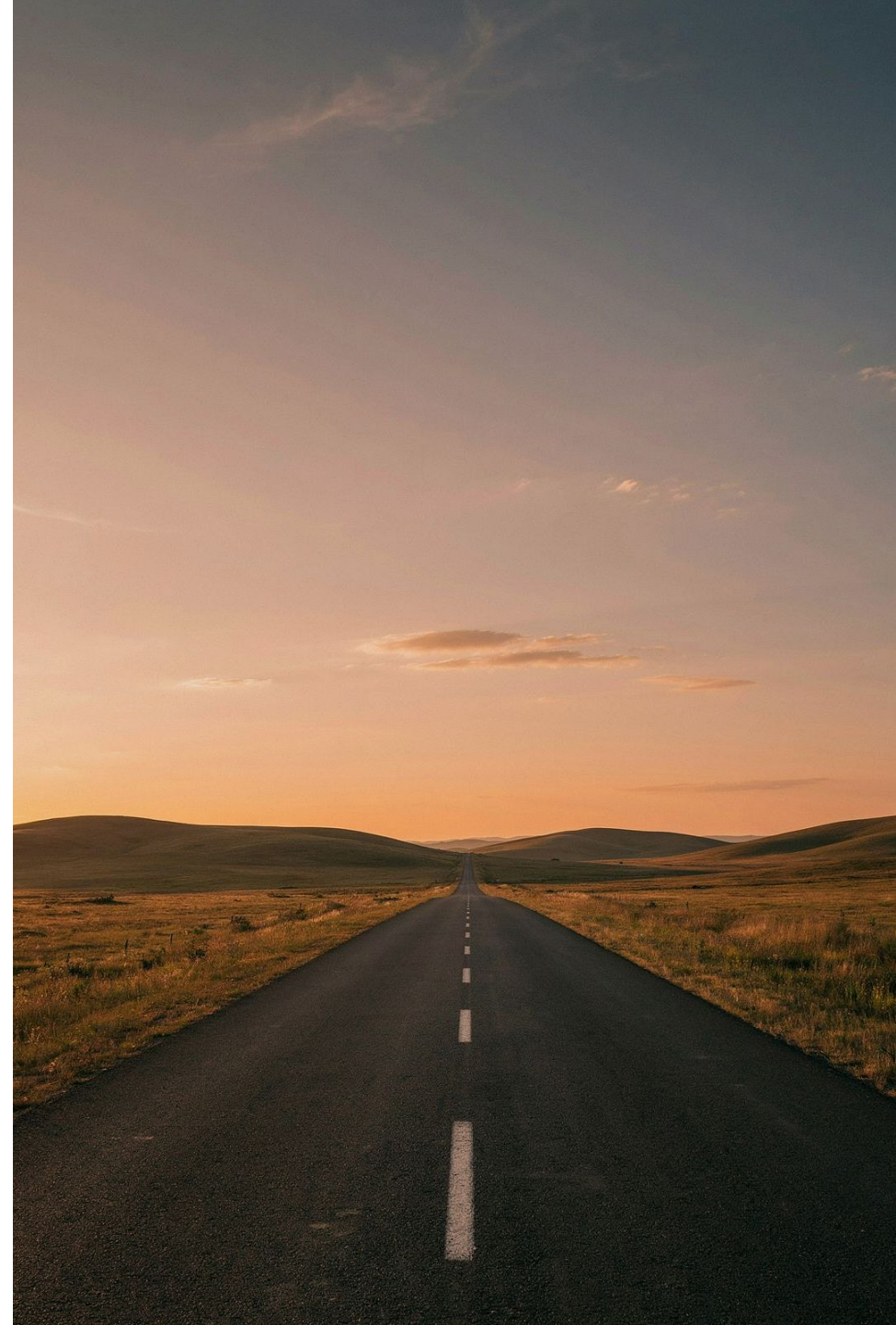
Experiment with schema compatibility modes in Registry

Integrate

Connect ksqlDB REST API to an application of your choice

Review

Revisit the full 4-day banking platform architecture



Thank You

Apache Kafka Day 4 — Stream Processing, ksqlDB, Schema Registry
& Confluent Control Center

Questions?

Open floor for discussion
and Q&A

Labs

Hands-on exercises available
in your lab environment

Resources

Confluent documentation & Kafka Streams API reference

